

Exercice

Q1

```
In [1]: def factoriel(n):
        f=1
        for i in range(2,n+1):
            f=f*i
        return f
```

Q2

```
In [3]: def calcul(x, k):
        return (-1)**k*x**(2*k)/factoriel(2*k)
```

Q3

```
In [ ]: O(n)
```

Q4

```
In [5]: def cosinus(x, n):
        s = 0
        for k in range(0,n):
            s+= calcul(x, k)
        return s
```

Partie I : Calcul numérique

Q1

```
In [1]: import matplotlib.pyplot as plt
        import numpy as np
```

```
In [19]: def RK4(f, t0, T, y0, N):
        h = T/N
        tt = np.linspace(t0, t0+T, N+1)
        Y = np.zeros((N+1,len(y0)))
        Y[0] = y0
        for i in range(len(Y)-1):
            k1 = f(tt[i], Y[i, :])
            k2 = f(tt[i] + h/2, Y[i] + k1*h/2)
            k3 = f(tt[i] + h/2, Y[i] + k2*h/2)
            k4 = f(tt[i] + h, Y[i, :] + k3*h)
            Y[i+1, :] = Y[i, :] + (h/6)*(k1 + 2*(k2+k3) + k4)
        return tt, Y
```

Q2.a

```
In [ ]: NT = X[0]+X[1]+X[2]+X[3]
z0 = -a*X[0]*X[2] + p*NT - k*X[0]
z1 = a*X[0]*X[2] - (b+k)*X[1]
z2 = b*X[1] - (c+k)*X[2]
z3 = c*X[2] - k*X[3]
```

Q2.b

```
In [20]: p = 0.004
a = 0.8
b = 0.5
c = 0.09
k = 0.005
```

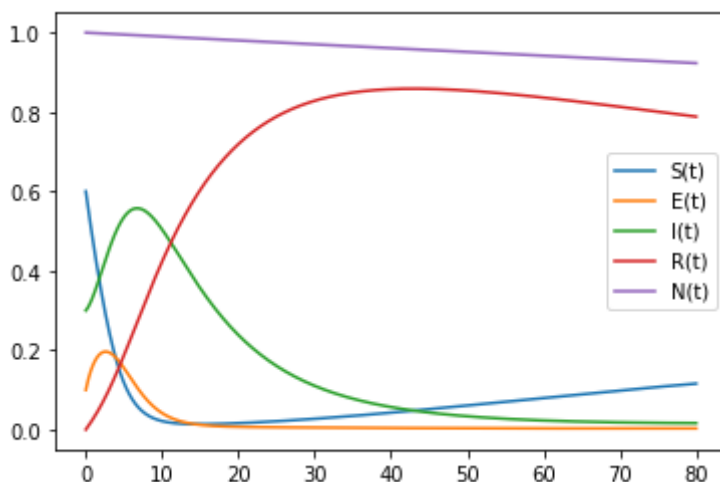
```
In [21]: def f(t, X):
    S, E, I, R = X
    z0 = -a*S*I + p*(S+E+I+R) - k*S
    z1 = a*S*I - (b+k)*E
    z2 = b*I - (c+k)*I
    z3 = c*I - k*R
    Z = np.array([z0, z1, z2, z3])
    return Z
```

Q3

```
In [22]: N = 10**3
tt, Y = RK4(f, 0, 80, [0.6, 0.1, 0.3, 0], N)
```

```
In [27]: plt.plot(tt, Y)
plt.plot(tt, Y[:, 0]+Y[:, 1]+Y[:, 2]+Y[:, 3])
plt.legend(("S(t)", "E(t)", "I(t)", "R(t)", "N(t)"))
```

```
Out[27]: <matplotlib.legend.Legend at 0x275c8482af0>
```



Partie II

```
In [ ]: insert into quotidien values('2020-09-17', 2513, 2847, 65, 'TXL')
```

```
In [ ]: select count(*) from quotidien where jour='2020-08-27' and infecté>=10000
```

```
In [ ]: select infectés, rétablis, décédés
from quotidiens join villes on codV=codVille
join pays on codP=codPays
where jour like '2020-09-%' and nom='Italie'
```

```
In [ ]: select codVille, nom, 100000*sum(infectes)/population as T1
from villes join quotidiens on codV=codVille
where jour like '2022-%'
group by codVille
Having T1 between 500 et 1000
order by T1
```

```
In [ ]: select pays.nom, 100*sum(infectes)/sum(retablis) as TG
from pays join on codP=codPay join villes on codV=codVille
group by codPays
Having TG>50
order by TG desc
```

```
In [ ]: select codPays, max(TM), min(TM), avg(TM)
from (select codeVille, codPays, 100*sum(decedes)/sum(infectes) as TM
from villes join quotidiens on codV=codeVille group by codVille)
group by codePays
order by count(codVille)
```

Partie III

```
In [24]: from math import sqrt

def distance(A, B):
    return sqrt((B[0]-A[0])**2 + (B[1]-A[1])**2)
```

```
In [25]: def plus_proches(T):
    A,B = T[0], T[1]
    d = distance(A,B)
    n = len(T)
    for i in range(n):
        for j in range(i+1, n):
            k = distance(T[i], T[j])
            if k<d :
                d = k
                A, B = T[i], T[j]
    return A, B
```

```
In [ ]: Compléxité quadratique O(n^2)
```

```
In [ ]: def tri_points(P, k):
```

```

n = len(P)
if n<=1 : return P
A,B = [],[]
for i in range(1, n):
    if P[i][k] < P[0][k] :
        A.append(P[i])
    else :
        B.append(P[i])
return tri_points(A, k) + [P[0]] + tri_points(B, k)

```

```

In [ ]: def bande_vertical(Y, x0, d):
        V=[]
        for t in Y:
            if x0-d<=t[0]<=x0+d :
                V.append(t)
        return V

```

```

In [ ]: def plus_proche_bande(B):
        k = distance(B[0], B[1])
        P,Q = B[0], B[1]
        n = len(B)
        for i in range(n):
            for j in range(i+1, min(n,i+8)):
                d = distance(B[i], B[j])
                if d<k:
                    k = d
                    P, Q = B[i], B[j]
        return P,Q

```

```

In [ ]: def plus_proche_rec(X, Y):
        n=len(X)
        if n<3 : return plus_proche_bande(X)
        X0 = X[n//2][0]
        Xg = X[:n//2]
        Xd = X[n//2:]
        A, B = plus_proche_rec(Xg, Y)
        d1 = distance(A,B)
        C, D = plus_proche_rec(Xd, Y)
        d2 = distance(C, D)
        if d1<d2 : E,F,d = A,B,d1
        else : E,F,d = C,D,d2
        B = bande_vertical(Y,X0,d)
        if len(B)<=1 : return E,F
        p,q = plus_proche_bande(B)
        if distance(p,q)<d : return p,q
        else : return E,F

```

```

In [ ]: O(nlog(n))

```